

دليل الأمثلة والنكشات الامتحانية في بايثون (Python Exam Traps)

شرح تفصيلي مبسط لـ 13 فخاً برمجياً متكرراً في الامتحانات

الجزء الأول: الأمثلة الأساسية والشرح المبسط (1 إلى 6)

1. تعديل قائمة مخبأة داخل Tuple

```
data = (1, [2, 3], 4)
data[1].append(5) # (1, [2, 3, 5], 4)
الـ Tuple ثابت، لكن عنصره الثاني List قابلة للتعديل! append() تعديلها بنجاح.
```

3. التعديل على Dict أثناء حلقة For

```
san = {"a": 10, "b": 20, "c": 30}
for k in san:
    if k == "b": san[k] = 200
print(sum(san.values())) # 240
تغيير قيمة (Value) مفتاح موجود أثناء الحلقة مسموح بدون أخطاء.
```

5. المتغير المرجعي للقاموس داخل القوائم

```
autot = [{"brand": "Ford"}, ...]
x = autot[0]; x["year"] = 2025
print(autot[0]["year"]) # 2025
x يشير مباشرة للقاموس الأصلي داخل القائمة، وأي تعديل يغيره فوراً.
```

2. الفروق بين Set والـ Dict والمراجع

```
s1 = {1, 2, 3}; s2 = s1; s2.add(4)
()a = {}; b = set
print(len(s1), type(a) == type(b)) # 4 False
s2 مرجع لنفس الذاكرة. و {} تعني Dict وليست Set فارغة.
```

4. متغير مستقل من تفكيك Tuple

```
tup = 1, 2, (3, 4)
x, y, z = tup; z = 100
print(tup[2][0]) # 3
تغيير المتغير الجانبي z لا يؤثر مطلقاً على الـ Tuple الأصلي tup.
```

6. المساواة المباشرة مقابل copy()

```
()a = [1, 2, 3]; b = a; c = a.copy()
a[0] = 99; b[1] = 88; c[2] = 77
print(a) # [99, 88, 3]
b = a تعديل مشترك. بينما copy() تصنع نسخة جديدة مستقلة تماماً.
```

7. سلوك دالة `setdefault()`

```
san = {"a": 1, "b": 2}
# x = san.setdefault("c", 3) # يضيف ويسجل 3
# y = san.setdefault("a", 99) # موجود، يتجاهل 99 ويرجع 1
print(x, y, len(san)) # 3 1 3
```

9. المראה الحية `keys()`

```
san = {"a": 10, "b": 20}
keys = san.keys(); san["c"] = 30
print(len(keys)) # 3
keys ( ) مرآة حية ترتبط بالقاموس وتحدث تلقائياً عند أي تغيير.
```

11. حذف عناصر أثناء حلقة `For`

```
luvut = [10, 20, 30, 40]
for x in luvut:
    if x == 10: luvut.remove(x)
print(luvut) # [20, 30, 40] (لا خطأ كود)
```

13. غياب المفتاح: `KeyError` مقابل `get()`

```
san = {"a": 1}
print(san["b"]) # يعطي KeyError ويدمر البرنامج!
print(san.get("b")) # يطبع None بسلام بدون خطأ.
استخدم [ ] لو متأكد من الوجود، واستخدم .get ( ) لتفادي انهيار البرنامج.
```

ملخص سريع للحفظ البصري:

- `List, Set, Dict` → قابلة للتعديل (Mutable).
- `Tuple, Int, String` → ثوابت (Immutable).
- `a = b` → مرجع مشترك. `a = b.copy()` → نسخة جديدة مستقلة.
- `{ }` → قاموس فارغ وليست مجموعة، بينما `set ( )` مجموعة فارغة.

8. القيمة الافتراضية لقائمة بالدالة

```
:def add(val, lst=[])
    lst.append(val); return lst
print(add(1)); print(add(2)) # [1, 2]
القائمة الافتراضية lst=[] تُخلق مرة واحدة فقط عند تعريف الدالة!
```

10. المقارنة بين `is` و `==`

```
a = [1, 2]; b = [1, 2]
print(a == b, a is b) # True False
== تقارن القيم، و is تقارن الموقع بالذاكرة (مختلف للقوائم).
```

12. إنشاء `Tuple` من عنصر واحد

```
x = (5) # int عادي
y = (5,) # Tuple حقيقي
print(type(x) == type(y)) # False
```